

Senior Full Stack Java Technical Specialist – React.js

Scenario-Based Interview Questions & Solutions | Tech Lead / 11+ Years

Prepared from the supplied job description. This guide contains 240 scenario-based interview questions (20 per topic) covering Java, Spring Boot, Microservices/REST, Kafka, React/TypeScript, AWS, databases, testing, CI/CD, system design, financial-crime case management, and Tech Lead responsibilities.

How to use this guide: For each scenario, answer in this order: clarify the problem → identify risks → explain the design/diagnosis → mention trade-offs → describe testing/monitoring → give the expected business outcome.

1. Core Java & Modern Java

20 scenario questions

1. Scenario / Question: A payment API suddenly shows high CPU after a release. How would you investigate?

Expected solution: Check thread/CPU metrics, capture thread dumps, identify hot methods, compare the release, inspect GC and allocation, reproduce with profiling, then fix and load-test. Avoid guessing from CPU alone.

2. Scenario / Question: A HashMap-based cache becomes inconsistent under concurrent requests. What would you do?

Expected solution: HashMap is not safe for concurrent mutation. Use ConcurrentHashMap or a properly synchronized cache, define atomic operations such as computeIfAbsent, and verify visibility/race conditions with concurrency tests.

3. Scenario / Question: A service processes 50,000 records and memory keeps increasing. How do you troubleshoot?

Expected solution: Inspect heap usage and GC, take heap dumps, identify retained objects, collections/caches/list growth, thread-local leaks and unclosed resources. Fix the retention path and validate with repeated load tests.

4. Scenario / Question: You need an immutable transaction object used by many threads. How would you design it?

Expected solution: Make the class final, fields private/final, initialize through constructor, validate inputs, avoid setters, and defensively copy mutable inputs/outputs.

5. Scenario / Question: Two threads update the same account balance. How do you prevent lost updates?

Expected solution: Use a transaction boundary and database locking/optimistic versioning for persistence. In-memory synchronization alone is insufficient across service instances.

6. Scenario / Question: A Java service has long GC pauses in production. What is your approach?

Expected solution: Measure allocation rate, heap occupancy and pause times; inspect GC logs/JFR; check heap sizing and object churn; select/tune an appropriate collector only after evidence; load-test the change.

7. Scenario / Question: When would you use CompletableFuture in a backend API?

Expected solution: For independent I/O-bound work that can safely run concurrently. Use bounded executors, timeouts, exception handling and cancellation; do not create unbounded threads.

8. Scenario / Question: A stream pipeline is slower than a loop for a hot path. What do you do?

Expected solution: Profile first. Streams improve readability but are not automatically faster. For a proven hot path, use a simple loop or optimize allocations while retaining correctness.

9. Scenario / Question: How would you safely handle null/optional data from an upstream API?

Expected solution: Define a clear contract, validate at the boundary, use Optional where it communicates absence, avoid Optional fields unnecessarily, and return meaningful error responses.

10. Scenario / Question: A singleton object contains mutable state and causes cross-request data leakage. Fix?

Expected solution: Make shared components stateless. If state is required, scope it correctly or isolate it per request/operation and use thread-safe structures where genuinely needed.

11. Scenario / Question: How would you implement retry without causing duplicate banking transactions?

Expected solution: Use idempotency keys and durable transaction state. Retry only transient failures with bounded exponential backoff/jitter; never blindly retry non-idempotent operations.

12. Scenario / Question: A production bug occurs only under concurrent load. How do you reproduce it?

Expected solution: Create a deterministic concurrency test where possible, stress with controlled parallelism, add correlation/diagnostic logging, use thread dumps/JFR and test shared-state boundaries.

13. Scenario / Question: What Java design pattern would you use for multiple payment providers?

Expected solution: Strategy is a natural fit: define a provider interface and separate implementations. Factory/registry can select the strategy; dependency injection keeps it testable.

14. Scenario / Question: How do you prevent resource leaks when calling external systems?

Expected solution: Use try-with-resources for AutoCloseable resources, configure connection/read timeouts, close response bodies, and monitor pool utilization.

15. Scenario / Question: A legacy utility method is static and difficult to unit test. What would you change?

Expected solution: Move external dependencies behind interfaces and inject them. Keep pure functions static if useful; avoid static state and wrap legacy calls behind testable adapters.

16. Scenario / Question: How do you choose between synchronized, Lock and atomic classes?

Expected solution: Use the simplest mechanism matching the problem. Atomics suit single-variable state transitions; locks suit compound critical sections; synchronized is often the clearest default.

17. Scenario / Question: What is the risk of parallelStream in a backend service?

Expected solution: It uses the common ForkJoinPool by default, which can contend with unrelated work and overwhelm downstream systems. Prefer explicit bounded executors for controlled concurrency.

18. Scenario / Question: How do you review Java code for production readiness?

Expected solution: Check correctness, concurrency, error handling, security, observability, resource lifecycle, performance, testability, API compatibility and operational rollback.

19. Scenario / Question: How would you migrate a Java 8 service to a newer JDK?

Expected solution: Inventory dependencies and runtime assumptions, upgrade incrementally, run unit/integration/security tests, inspect removed/changed APIs, performance-test, then deploy gradually.

20. Scenario / Question: What would you monitor in a high-throughput Java service?

Expected solution: Latency percentiles, throughput, error rate, saturation, CPU, memory/GC, thread pools, connection pools, downstream latency, queue depth and business transaction metrics.

2. Spring Boot, Spring & Spring Batch

20 scenario questions

21. Scenario / Question: A Spring Boot service starts slowly after many dependencies are added. How do you improve startup?

Expected solution: Measure startup phases, inspect auto-configuration and bean creation, remove unused starters/configuration, defer noncritical initialization where appropriate, and validate readiness separately from process startup.

22. Scenario / Question: A bean unexpectedly has shared state between requests. What is likely wrong?

Expected solution: The default singleton scope is shared. Make the bean stateless or use an appropriate scope; never store request-specific mutable state in a singleton.

23. Scenario / Question: Two implementations of an interface exist and Spring injects the wrong one. Fix?

Expected solution: Use `@Qualifier` or `@Primary` and make the selection explicit. Avoid ambiguous implicit wiring.

24. Scenario / Question: A REST endpoint occasionally times out while downstream is slow. How would you design it?

Expected solution: Set connect/read timeouts, use circuit breaking/bulkheads where appropriate, propagate correlation IDs, return bounded failures and avoid tying up request threads indefinitely.

25. Scenario / Question: How would you implement global error handling?

Expected solution: Use `@RestControllerAdvice` with consistent error codes/messages, validation handling, correlation ID and safe diagnostics. Do not expose stack traces or sensitive data.

26. Scenario / Question: A transaction rolls back unexpectedly because an exception is caught. Explain.

Expected solution: Spring transaction rollback behavior depends on exception type and transaction proxy boundaries. Do not swallow exceptions unintentionally; configure rollback rules when needed and understand proxy/self-invocation limitations.

27. Scenario / Question: A `@Transactional` method calls another `@Transactional` method in the same class. Is the second annotation guaranteed to apply?

Expected solution: Not through normal proxy-based interception: self-invocation bypasses the proxy. Move the method to another bean or restructure the transaction boundary.

28. Scenario / Question: How would you process 10 million records with Spring Batch?

Expected solution: Use chunk-oriented processing, paging/cursor readers as appropriate, batching writes, controlled transaction size, restartability, skip/retry policies and monitoring. Partition only when parallelism is justified.

29. Scenario / Question: A batch job restarts and duplicates records. How do you make it restartable?

Expected solution: Use stable item identity, job/step execution metadata, idempotent writes or checkpoints, appropriate restart configuration and unique constraints where useful.

30. Scenario / Question: A batch job is slower after increasing chunk size. Why?

Expected solution: Large chunks may increase transaction duration, memory, lock contention and rollback cost. Benchmark different sizes against DB behavior and downstream constraints.

31. Scenario / Question: How do you prevent one bad record from failing a whole batch?

Expected solution: Use skip policies for known recoverable data errors, dead-letter/error storage and monitoring. Do not skip systemic failures such as database outages.

32. Scenario / Question: How would you expose health endpoints safely?

Expected solution: Use Spring Boot Actuator with restricted exposure, authentication/network controls and readiness/liveness semantics. Do not expose sensitive environment/config endpoints publicly.

33. Scenario / Question: How do you externalize configuration across environments?

Expected solution: Use environment-specific configuration/secrets management, environment variables or a configuration service. Keep secrets out of source control.

34. Scenario / Question: A Spring service has circular dependencies. How do you fix them?

Expected solution: Prefer redesigning responsibilities and introducing an abstraction or mediator. Avoid relying on circular-reference settings as a structural fix.

35. Scenario / Question: How would you design a reusable Spring Boot library for bank teams?

Expected solution: Define stable APIs, minimal transitive dependencies, configuration properties, extension points, documentation, compatibility policy, semantic versioning and automated tests.

36. Scenario / Question: How do you test Spring Boot controllers and services?

Expected solution: Use unit tests for business logic with Mockito where useful; use focused MVC tests for controllers and integration tests for serialization, security, persistence and external boundaries.

37. Scenario / Question: A service receives duplicate requests. Where should idempotency be implemented?

Expected solution: At the business/API boundary using an idempotency key persisted with operation status/result, backed by a unique constraint to make concurrent duplicates safe.

38. Scenario / Question: How do you handle graceful shutdown?

Expected solution: Stop accepting new traffic, allow in-flight work to finish within a deadline, drain consumers where applicable, close resources and expose readiness so traffic is removed before termination.

39. Scenario / Question: How would you secure secrets in Spring Boot?

Expected solution: Use a managed secrets store and inject at runtime. Avoid logging secrets and configuration dumps; rotate credentials and restrict access by workload identity.

40. Scenario / Question: A production Spring Boot service has intermittent 5xx errors. What is your investigation path?

Expected solution: Correlate request IDs with logs/traces/metrics, identify error class and downstream dependency, inspect recent deployments/config changes, reproduce, mitigate safely, then fix and add regression tests.

3. Microservices, REST API & API Gateway

20 scenario questions

41. Scenario / Question: Design a high-volume account transaction API. What are the key concerns?

Expected solution: Clear resource/API contracts, idempotency, authorization, validation, transactional consistency, timeouts, observability, rate limiting and horizontal scalability.

42. Scenario / Question: A client retries POST after a network timeout. How do you avoid duplicate transactions?

Expected solution: Require an idempotency key, persist operation state atomically, return the original result for the same key and reject conflicting payloads.

43. Scenario / Question: One microservice is unavailable. How should dependent services behave?

Expected solution: Use timeouts, circuit breaker, bulkhead, fallback only when business-safe, and asynchronous decoupling where suitable. Avoid cascading failure.

44. Scenario / Question: An API has p99 latency spikes but average latency is normal. What do you investigate?

Expected solution: Look at traces and percentile latency, slow downstream calls, DB queries, GC, thread/connection pools, retries and traffic patterns. Average latency can hide tail problems.

45. Scenario / Question: How would you version a public REST API?

Expected solution: Prefer backward-compatible evolution; introduce a new version only for breaking changes, define deprecation timelines and contract tests. Keep versioning consistent across gateway and services.

46. Scenario / Question: How do you design pagination for millions of records?

Expected solution: Prefer cursor/keyset pagination for large changing datasets; define stable ordering and opaque cursors. Offset pagination can become expensive and inconsistent at scale.

47. Scenario / Question: A gateway becomes a bottleneck. What would you do?

Expected solution: Measure CPU/network/connection saturation, scale horizontally, optimize filters/auth calls, use caching where safe and ensure downstream services are also healthy.

48. Scenario / Question: How would you implement API rate limiting?

Expected solution: Define limits per client/user/API, use gateway/distributed rate-limiting, return 429 with retry guidance and monitor rejected traffic.

49. Scenario / Question: How do you validate an API contract across teams?

Expected solution: Use OpenAPI/schema definitions, consumer/provider contract tests, compatibility checks in CI and versioned API artifacts.

50. Scenario / Question: An API returns sensitive information in an error response. What do you change?

Expected solution: Return safe error codes/messages, log detailed diagnostics securely, redact PII/secrets, standardize error handling and add security tests.

51. Scenario / Question: How would you secure service-to-service communication?

Expected solution: Use strong workload identity/mTLS or signed tokens as appropriate, authorize scopes/roles, rotate credentials and audit access.

52. Scenario / Question: What is the difference between authentication and authorization in an API?

Expected solution: Authentication establishes who/what the caller is; authorization determines what that identity may do. Both must be enforced at appropriate boundaries.

53. Scenario / Question: A downstream API is slow and retries increase traffic. Diagnose?

Expected solution: Retry amplification. Use strict timeouts, bounded retries with jitter, circuit breaking and concurrency limits. Prefer not retrying permanent errors.

54. Scenario / Question: How would you handle partial failure in an orchestration of three services?

Expected solution: Define compensating actions or a saga where appropriate, persist workflow state, make steps idempotent and expose clear final status.

55. Scenario / Question: How do you design backward compatibility for shared APIs?

Expected solution: Add fields rather than changing/removing existing semantics, tolerate unknown fields where safe, keep old contracts working and use consumer contract tests.

56. Scenario / Question: What belongs in an API Gateway versus a microservice?

Expected solution: Gateway concerns include routing, edge authentication, rate limits and common cross-cutting policies. Business rules should remain in domain services.

57. Scenario / Question: How would you trace a request across 8 microservices?

Expected solution: Propagate a correlation/trace ID and use distributed tracing with structured logs and consistent service metadata.

58. Scenario / Question: How do you protect an API from oversized requests?

Expected solution: Set gateway/server body limits, validate payload sizes/content types, reject early and monitor abnormal clients.

59. Scenario / Question: When would you choose synchronous REST versus asynchronous messaging?

Expected solution: REST for request/response interactions needing immediate result; messaging for decoupling, buffering, long-running workflows and event propagation.

60. Scenario / Question: How would you safely deploy a breaking API change?

Expected solution: Introduce a compatible version, migrate consumers, run contract tests, observe traffic, communicate deprecation, then remove the old version after a defined window.

4. Kafka & Event-Driven Architecture

20 scenario questions

61. Scenario / Question: A Kafka consumer processes the same payment event twice. How do you make processing safe?

Expected solution: Assume at-least-once delivery. Use idempotent consumers with a durable event/business key and unique constraints or processed-event state.

62. Scenario / Question: Consumer lag keeps increasing. What do you investigate?

Expected solution: Producer rate, consumer throughput, partition count, consumer group health, rebalances, downstream latency, DB locks, thread pools and errors/retries.

63. Scenario / Question: How do you choose Kafka partition keys for transaction events?

Expected solution: Choose a key that preserves required ordering, often account/customer/transaction aggregate ID, while avoiding hot partitions.

64. Scenario / Question: A consumer crashes after DB commit but before Kafka offset commit. What happens?

Expected solution: The message can be redelivered. Design the DB operation idempotently or use an appropriate transactional/outbox architecture.

65. Scenario / Question: How would you guarantee an event is published when a DB transaction commits?

Expected solution: Use the transactional outbox pattern: write business state and outbox event in one DB transaction, then publish asynchronously with reliable retry.

66. Scenario / Question: How do you handle poison messages?

Expected solution: Bound retries, move permanently failing messages to a dead-letter topic with diagnostic metadata, alert and provide a controlled replay process.

67. Scenario / Question: A topic has too few partitions for required throughput. What is the impact?

Expected solution: Consumer parallelism is limited by partition count. Increase partitions carefully, considering key distribution and ordering requirements.

68. Scenario / Question: Why can increasing consumer instances fail to improve throughput?

Expected solution: There may be fewer partitions, downstream bottlenecks, DB contention, network saturation or inefficient processing.

69. Scenario / Question: How do you preserve ordering in Kafka?

Expected solution: Ordering is guaranteed within a partition, not globally. Route related events to the same partition key.

70. Scenario / Question: How would you handle schema evolution?

Expected solution: Use a schema contract/registry where appropriate, backward/forward compatibility rules, versioned changes and consumer tests.

71. Scenario / Question: A consumer rebalance causes latency spikes. How do you reduce it?

Expected solution: Avoid long processing in poll loops, tune consumer settings carefully, control max processing time, improve handler speed and investigate unstable membership.

72. Scenario / Question: How would you replay historical events safely?

Expected solution: Use a separate consumer group or controlled replay pipeline, ensure idempotency, isolate side effects and monitor throughput.

73. Scenario / Question: When should Kafka not be used?

Expected solution: When simple synchronous request/response is enough, strict immediate consistency is required without an asynchronous boundary, or operational complexity outweighs benefits.

74. Scenario / Question: How do you monitor Kafka in production?

Expected solution: Track consumer lag, throughput, broker health, under-replicated partitions, request latency, errors, rebalances and dead-letter volume.

75. Scenario / Question: How would you design an audit event stream?

Expected solution: Emit immutable events with event ID, actor/service, timestamp, action, entity reference and correlation ID; protect integrity/access and define retention.

76. Scenario / Question: How do you handle a temporary downstream DB outage in a Kafka consumer?

Expected solution: Pause/back off consumption or bound concurrency, retry transiently, avoid hot-looping, preserve messages and use DLQ only for non-recoverable cases.

77. Scenario / Question: What is the role of a consumer group?

Expected solution: It distributes partitions among consumers so each partition is processed by one consumer within the group, enabling horizontal scaling.

78. Scenario / Question: How would you prevent duplicate side effects from event retries?

Expected solution: Use idempotency keys, transactional state transitions, unique constraints and/or a processed-event table.

79. Scenario / Question: How do you choose retention for transaction events?

Expected solution: Based on business replay needs, regulatory/audit requirements, storage cost and downstream recovery strategy.

80. Scenario / Question: What would you include in an event payload?

Expected solution: Stable event type/version, event ID, aggregate/entity ID, occurred-at time, correlation ID and business data required by consumers; avoid unnecessary sensitive data.

5. React.js & TypeScript

20 scenario questions

81. Scenario / Question: A React page renders slowly when a transaction table grows to 20,000 rows. What do you do?

Expected solution: Use server-side pagination/filtering, virtualization for large lists, memoization where measured useful, stable keys and avoid unnecessary parent re-renders.

82. Scenario / Question: A child component re-renders on every parent update. How do you investigate?

Expected solution: Use React DevTools profiler, inspect changing props/function references, use `React.memo/useMemo/useCallback` selectively and fix unnecessary state placement.

83. Scenario / Question: A `useEffect` triggers repeated API calls. Why and how do you fix it?

Expected solution: Inspect dependency array and state updates inside the effect. Stabilize dependencies, separate effects by responsibility and avoid setting state that re-triggers the same effect unnecessarily.

84. Scenario / Question: How would you handle API loading/error/success states?

Expected solution: Model explicit states rather than ambiguous flags, show loading feedback, safe errors and retry behavior; cancel/ignore stale requests when needed.

85. Scenario / Question: A user submits a form twice. How do you prevent duplicate transactions?

Expected solution: Disable or lock submission while pending, use client-side request state, but rely on backend idempotency as the real guarantee.

86. Scenario / Question: How would you secure a React banking UI?

Expected solution: Do not trust client-side authorization; enforce permissions server-side, protect tokens appropriately, prevent XSS, validate inputs, use secure headers and avoid storing sensitive data unnecessarily.

87. Scenario / Question: How do you share state across many React components?

Expected solution: Start with local state; lift state for related components; use Context for suitable cross-cutting state; use a state library when complexity and update patterns justify it.

88. Scenario / Question: What TypeScript benefit matters most in a large banking frontend?

Expected solution: Compile-time contracts, safer refactoring, explicit domain models and fewer runtime type errors. Keep API models aligned with backend contracts.

89. Scenario / Question: An API response changes and breaks the UI. How do you prevent it?

Expected solution: Use typed API clients/schemas, contract tests, backward-compatible APIs and CI checks; handle optional/new fields safely.

90. Scenario / Question: How would you implement route protection?

Expected solution: Use authentication state and route guards at the UI level for UX, while server-side authorization remains authoritative.

91. Scenario / Question: A React component has a huge amount of logic. How would you refactor?

Expected solution: Separate presentation from domain/data logic, extract hooks/services, create focused components and test business logic independently.

92. Scenario / Question: How do you handle cancellation of stale API requests?

Expected solution: Use AbortController or an equivalent request lifecycle mechanism, ignore stale responses and clean up effects.

93. Scenario / Question: What are good practices for reusable React components?

Expected solution: Clear props, composability, accessibility, controlled behavior, minimal hidden state, documented contracts and tests.

94. Scenario / Question: How do you test a transaction form?

Expected solution: Test user behavior: rendering, validation, submission, loading, errors, success and accessibility. Mock network boundaries rather than implementation details.

95. Scenario / Question: A React app exposes environment secrets in its bundle. Is that safe?

Expected solution: No. Frontend build-time variables are generally public. Keep secrets server-side and expose only non-sensitive configuration.

96. Scenario / Question: How do you optimize a large bundle?

Expected solution: Analyze bundle composition, lazy-load routes/components, remove unused dependencies, tree-shake, compress and cache static assets.

97. Scenario / Question: How would you handle accessibility for a banking application?

Expected solution: Semantic HTML, keyboard navigation, labels, focus management, accessible errors, sufficient contrast and automated plus manual accessibility testing.

98. Scenario / Question: A dashboard shows inconsistent data due to race conditions between requests. Fix?

Expected solution: Cancel stale requests, track request identity/version, centralize data fetching and update state only for the current request.

99. Scenario / Question: How do you decide between Context and a state library?

Expected solution: Context is suitable for relatively stable cross-cutting values; complex shared state with selectors, caching, async workflows and frequent updates may justify a dedicated library.

100. Scenario / Question: How would you migrate a JavaScript React app to TypeScript?

Expected solution: Migrate incrementally, start with shared types/components, enable useful compiler checks gradually, type API boundaries and avoid a risky big-bang rewrite.

6. AWS / Cloud-Native Design

20 scenario questions

101. Scenario / Question: A service must handle a 10x traffic spike. How would you design for it on AWS?

Expected solution: Stateless horizontally scalable services, load balancing, autoscaling, caching where safe, asynchronous buffering, DB capacity planning and observability.

102. Scenario / Question: An EC2-based service becomes unhealthy during traffic peaks. What do you inspect?

Expected solution: CPU/memory, network, connection pools, application latency, load balancer health, autoscaling signals and downstream limits.

103. Scenario / Question: How would you secure a cloud-native microservice?

Expected solution: Least-privilege IAM, private networking where appropriate, managed secrets, encryption in transit/at rest, security groups, logging and workload identity.

104. Scenario / Question: A deployment works in one environment but fails in another. How do you prevent this?

Expected solution: Infrastructure/configuration as code, immutable artifacts, environment parity, automated tests and explicit configuration validation.

105. Scenario / Question: How would you design highly available services across AWS Availability Zones?

Expected solution: Distribute stateless compute across AZs, use managed HA data services where appropriate, health checks, automated failover and eliminate single points of failure.

106. Scenario / Question: How do you reduce cloud cost without hurting performance?

Expected solution: Measure utilization, right-size compute, autoscale, optimize storage/egress, use appropriate managed services and review idle resources.

107. Scenario / Question: A service loses connectivity to RDS. Investigation?

Expected solution: Check security groups/network routes/DNS, DB health, connection pool exhaustion, credentials/secrets, limits and application error patterns.

108. Scenario / Question: How would you handle secrets in AWS?

Expected solution: Use a managed secret store/KMS-backed solution, IAM least privilege, rotation and runtime retrieval; never bake secrets into images or source code.

109. Scenario / Question: What is a good deployment strategy for a critical banking API?

Expected solution: Blue/green or canary with automated health checks, rollback, database compatibility planning and gradual traffic shift.

110. Scenario / Question: How do you design observability in AWS?

Expected solution: Centralized structured logs, metrics, distributed traces, alarms tied to SLOs, dashboards and correlation IDs.

111. Scenario / Question: A serverless function times out. What do you investigate?

Expected solution: Duration breakdown, downstream latency, cold starts where relevant, memory allocation, network configuration, retries and concurrency limits.

112. Scenario / Question: How do you prevent a cloud outage from becoming a data integrity issue?

Expected solution: Transactional boundaries, durable storage, idempotency, retries with limits, backups, recovery procedures and tested failover.

113. Scenario / Question: When would you use a queue between microservices?

Expected solution: When buffering, decoupling, asynchronous processing or smoothing traffic is valuable. Define delivery semantics and idempotency.

114. Scenario / Question: How would you migrate a monolith to AWS microservices?

Expected solution: Strangler approach: identify bounded domains, extract low-risk services, introduce APIs/events, migrate data carefully, observe, and progressively retire monolith paths.

115. Scenario / Question: What cloud metrics should be part of an application SLO?

Expected solution: Availability, latency percentiles, error rate and throughput, plus dependency/saturation metrics relevant to the user journey.

116. Scenario / Question: How do you test disaster recovery?

Expected solution: Regularly execute recovery drills, measure RTO/RPO, validate backups, failover, DNS/configuration and application correctness after recovery.

117. Scenario / Question: A service is healthy but users receive errors through the load balancer. What do you check?

Expected solution: Listener/routing rules, target health, TLS, security controls, headers/timeouts and whether the health check validates only a shallow path.

118. Scenario / Question: How do you protect an API from abusive traffic in the cloud?

Expected solution: Gateway/WAF controls, rate limits, authentication, quotas, anomaly monitoring and autoscaling with downstream protection.

119. Scenario / Question: What is infrastructure as code and why is it useful?

Expected solution: Declarative infrastructure managed through version control and automation, enabling repeatability, review, drift control and consistent environments.

120. Scenario / Question: How do you approach cloud certification knowledge in an interview?

Expected solution: Explain trade-offs and operational reasoning rather than listing services; map service choice to availability, security, scalability, latency and cost.

7. Oracle, PostgreSQL, PL/SQL & NoSQL

20 scenario questions

121. Scenario / Question: A query that was fast in Oracle is slow after migration to PostgreSQL. How do you investigate?

Expected solution: Compare execution plans, indexes, statistics, data distribution, SQL semantics and parameter behavior; tune based on EXPLAIN ANALYZE rather than translating syntax blindly.

122. Scenario / Question: A transaction API gets deadlocks. What is your approach?

Expected solution: Capture deadlock details, identify lock order, keep transactions short, access resources in consistent order, add proper indexes and retry only when the operation is safe.

123. Scenario / Question: A table has 500 million rows. How would you design queries and indexes?

Expected solution: Understand access patterns, use selective indexes, partition only where justified, avoid unnecessary columns, use keyset pagination and validate with actual plans.

124. Scenario / Question: How would you migrate Oracle PL/SQL to PostgreSQL?

Expected solution: Inventory packages/procedures/functions/types, map incompatible features, rewrite logic, automate data validation and test performance and transaction semantics.

125. Scenario / Question: A query has an index but still scans the table. Why?

Expected solution: The planner may estimate the index is not selective, statistics may be stale, expression/type mismatch may prevent use, or scanning may genuinely be cheaper.

126. Scenario / Question: How do you avoid SQL injection?

Expected solution: Use parameterized queries/prepared statements, ORM bind parameters and strict validation. Never concatenate untrusted values into SQL.

127. Scenario / Question: How do you handle connection pool exhaustion?

Expected solution: Check pool size, leaked/unclosed connections, long queries/transactions, downstream latency and application concurrency. Increasing the pool blindly can overload the DB.

128. Scenario / Question: A DB migration causes unexpected numeric errors. What do you check?

Expected solution: Data types, invalid legacy values, precision/scale, null/empty strings, implicit casts and transformation rules; validate data before loading.

129. Scenario / Question: How would you design optimistic locking?

Expected solution: Add a version column and update with WHERE id=? AND version=?; if no row updates, report a concurrency conflict and retry only if business-safe.

130. Scenario / Question: When would you choose NoSQL over a relational DB?

Expected solution: When access patterns, scale, flexible schema or specific distribution needs fit NoSQL. Do not choose it merely because the dataset is large.

131. Scenario / Question: How do you maintain consistency across PostgreSQL and Kafka?

Expected solution: Use transactional outbox or another reliable integration pattern rather than trying to coordinate two independent commits.

132. Scenario / Question: How would you optimize a PL/SQL-heavy process?

Expected solution: Measure SQL versus procedural time, reduce row-by-row processing, use set-based operations, batch work and inspect execution plans.

133. Scenario / Question: A query works in development but times out in production. Why?

Expected solution: Different data volume/distribution, statistics, indexes, parameters, concurrency, hardware and plans. Compare plans and runtime metrics.

134. Scenario / Question: How do you safely add an index to a high-volume production table?

Expected solution: Estimate build impact, storage and write overhead; use an online/concurrent mechanism supported by the DB, schedule carefully and monitor locks.

135. Scenario / Question: How do you model audit data?

Expected solution: Append-oriented records with immutable event metadata, actor, timestamp, operation, entity reference, correlation ID and appropriate retention/access controls.

136. Scenario / Question: A report needs many joins and is slow. Would you denormalize?

Expected solution: First optimize query/indexes and workload. Consider read models/materialized views or denormalization when justified by stable reporting access patterns.

137. Scenario / Question: How do you handle database schema changes in CI/CD?

Expected solution: Version migrations, make changes backward-compatible across deployment phases, automate validation and define rollback/forward-fix strategy.

138. Scenario / Question: How do you test database changes?

Expected solution: Unit-test transformation logic and integration-test real DB behavior using representative schemas/data; verify constraints, plans and performance for critical paths.

139. Scenario / Question: A NoSQL document grows very large. What risks arise?

Expected solution: Read/write amplification, item/document limits, hot partitions and inefficient updates. Revisit aggregate boundaries and access patterns.

140. Scenario / Question: What DB metrics matter for a high-volume banking application?

Expected solution: Latency, throughput, locks, deadlocks, connection usage, CPU/IO, cache hit ratio, slow queries, replication lag and storage growth.

8. JUnit, Mockito, Integration Testing & Quality

20 scenario questions

141. Scenario / Question: A unit test passes but production fails at the DB boundary. What is missing?

Expected solution: Integration/contract testing. Unit tests isolate logic; integration tests validate serialization, DB behavior, transactions and external boundaries.

142. Scenario / Question: Mockito tests are heavily coupled to implementation details. How do you improve them?

Expected solution: Prefer behavior-focused assertions and verify only important interactions. Avoid excessive verify calls and mocking simple value objects.

143. Scenario / Question: A test suite takes 30 minutes. How do you reduce it?

Expected solution: Profile tests, parallelize safe tests, reduce unnecessary integration setup, use focused test slices and move expensive scenarios to appropriate pipeline stages.

144. Scenario / Question: How would you test retry logic?

Expected solution: Control a mocked/fake dependency to fail transiently then succeed, verify bounded attempts/backoff behavior and confirm permanent errors are not retried.

145. Scenario / Question: How do you test Kafka consumers?

Expected solution: Unit-test handler logic, integration-test serialization and broker behavior, test duplicate delivery, ordering assumptions and failure/DLQ paths.

146. Scenario / Question: How do you test a REST API contract?

Expected solution: Validate request/response schemas, status codes, headers and compatibility. Add consumer/provider contract tests for independently deployed services.

147. Scenario / Question: A test is flaky only in CI. What do you investigate?

Expected solution: Timing/race conditions, shared state, test order, external dependencies, resource limits and parallel execution. Remove sleeps and make synchronization deterministic.

148. Scenario / Question: How do you test concurrent Java code?

Expected solution: Use controlled concurrency tests, barriers/latches, repeated stress runs and assertions around invariants. Avoid tests that depend on timing alone.

149. Scenario / Question: How would you test idempotency?

Expected solution: Send the same request/event multiple times concurrently and sequentially; assert one business effect and consistent returned result.

150. Scenario / Question: How do you test database transaction rollback?

Expected solution: Force a failure after an intermediate write and assert that the transaction leaves no partial state.

151. Scenario / Question: What belongs in unit versus integration tests?

Expected solution: Unit tests are fast and isolated; integration tests validate component boundaries; end-to-end tests validate critical user journeys and should remain focused.

152. Scenario / Question: How would you use Mockito for external API clients?

Expected solution: Mock the client interface in service unit tests and test success, timeout, retryable error and permanent error behavior.

153. Scenario / Question: A developer has 90% line coverage but many bugs. Explain.

Expected solution: Coverage measures execution, not correctness. Missing assertions, edge cases, contract tests, concurrency tests and realistic scenarios can still leave major gaps.

154. Scenario / Question: How do you test security authorization?

Expected solution: Test allowed and denied roles/scopes, resource ownership, expired/invalid credentials and endpoint-level enforcement; include integration coverage.

155. Scenario / Question: How do you make tests maintainable across refactoring?

Expected solution: Test public behavior and contracts rather than private implementation details; use builders/fixtures carefully and keep tests readable.

156. Scenario / Question: How would you test a Spring Batch job?

Expected solution: Test step logic, reader/writer behavior, restartability, skip/retry, transaction boundaries and representative data volumes.

157. Scenario / Question: How do you test failure recovery in microservices?

Expected solution: Inject dependency failures/timeouts and verify circuit breaking, retries, fallback, idempotency and observability.

158. Scenario / Question: What quality gates would you put in CI?

Expected solution: Compilation, unit tests, static analysis, dependency/security scanning, coverage threshold where meaningful, contract/integration tests and artifact verification.

159. Scenario / Question: How do you review a pull request as a Tech Lead?

Expected solution: Check requirement alignment, design, correctness, security, performance, operational impact, tests, backward compatibility and maintainability.

160. Scenario / Question: How do you decide when to add an automated test?

Expected solution: Add tests for business-critical behavior, regressions, boundaries, failure modes and complex logic; prioritize tests that prevent expensive production failures.

9. Jenkins / CI-CD / DevOps / Git

20 scenario questions

161. Scenario / Question: A production deployment passes CI but fails at startup. How do you improve the pipeline?

Expected solution: Add environment-like integration tests, startup/readiness checks, configuration validation, smoke tests and deployment gates.

162. Scenario / Question: A pipeline takes 45 minutes. How do you optimize it?

Expected solution: Measure stage duration, parallelize independent tests, cache dependencies safely, reuse build artifacts and separate fast PR checks from slower full suites.

163. Scenario / Question: How would you implement zero/low-downtime deployment?

Expected solution: Use rolling, blue/green or canary deployment with readiness checks, backward-compatible DB changes and automated rollback.

164. Scenario / Question: A deployment introduces high error rates. What should happen?

Expected solution: Automated monitoring should detect the issue; stop/rollback traffic when safe, preserve evidence, identify root cause and add a regression test.

165. Scenario / Question: How do you manage database migrations in CI/CD?

Expected solution: Version migrations with the application, make schema changes backward compatible, run validation and deploy schema changes in safe phases.

166. Scenario / Question: What is your Git branching strategy for multiple teams?

Expected solution: Keep it simple: trunk-based or short-lived feature branches, protected main branch, mandatory reviews and automated checks. Choose based on release model.

167. Scenario / Question: A developer force-pushes shared history. How do you handle it?

Expected solution: Protect critical branches, educate on safe workflows, recover using relog where necessary and establish branch protection rules.

168. Scenario / Question: How do you handle secrets in Jenkins?

Expected solution: Use credential stores/secret managers, mask values, restrict permissions and prevent secrets from appearing in logs or artifacts.

169. Scenario / Question: How do you promote the same artifact across environments?

Expected solution: Build once, store immutable versioned artifact, then promote that exact artifact with environment-specific configuration.

170. Scenario / Question: How do you implement automated quality gates?

Expected solution: Fail builds for compile/test/security/quality requirements appropriate to the project and publish clear reports so failures are actionable.

171. Scenario / Question: What should a rollback strategy contain?

Expected solution: Known-good artifact/version, compatible schema strategy, traffic reversal, health checks, ownership and a tested recovery procedure.

172. Scenario / Question: A pipeline intermittently fails due to dependency downloads. Fix?

Expected solution: Use repository caching/proxy, pin versions, retry transient network operations carefully and monitor dependency availability.

173. Scenario / Question: How do you manage feature flags?

Expected solution: Use controlled configuration, default-safe behavior, ownership, expiry dates and auditability; avoid accumulating permanent flags.

174. Scenario / Question: How do you secure CI/CD?

Expected solution: Least-privilege service accounts, isolated runners, protected credentials, dependency scanning, signed/verified artifacts and audit logs.

175. Scenario / Question: How do you automate API tests in CI?

Expected solution: Run deterministic contract/integration tests against an ephemeral or controlled environment, publish reports and fail on critical regressions.

176. Scenario / Question: What is deployment drift?

Expected solution: Runtime infrastructure/configuration differs from declared/versioned configuration. Infrastructure as code and drift detection reduce it.

177. Scenario / Question: How do you handle hotfixes while maintaining mainline stability?

Expected solution: Create a controlled hotfix from the released baseline, test it, deploy with approvals, then merge the fix back into the main development line.

178. Scenario / Question: What Git practices improve code quality?

Expected solution: Small focused commits, meaningful messages, pull requests, reviews, protected branches, automated checks and avoiding long-lived branches.

179. Scenario / Question: How do you measure DevOps effectiveness?

Expected solution: Deployment frequency, lead time, change failure rate, recovery time, pipeline reliability and production quality.

180. Scenario / Question: A build is green but artifact contents are wrong. How do you prevent this?

Expected solution: Artifact validation, reproducible builds, checksums/signing, SBOM/dependency verification and deployment of the exact tested artifact.

10. System Design, Scalability, Availability & Performance

20 scenario questions

181. Scenario / Question: Design a high-volume transaction monitoring platform. What components would you propose?

Expected solution: API gateway, stateless microservices, event ingestion via Kafka, rules/decision services, transactional DB, scalable read model/search where justified, case management service, audit trail and observability.

182. Scenario / Question: How would you design for 99.99% availability?

Expected solution: Remove single points of failure, multi-AZ deployment, health checks, automated failover, durable data, graceful degradation, capacity headroom and tested DR.

183. Scenario / Question: A service can handle 1,000 TPS but needs 10,000 TPS. What is your approach?

Expected solution: Identify bottleneck using profiling/load tests, scale stateless tiers, partition work, batch/buffer with Kafka, optimize DB access and validate downstream capacity.

184. Scenario / Question: How do you calculate capacity before launch?

Expected solution: Estimate peak/average traffic, payload size, concurrency, processing time, DB operations, growth and headroom; validate assumptions with load tests.

185. Scenario / Question: A database is the bottleneck in a microservice system. What options do you consider?

Expected solution: Query/index optimization, caching, batching, read replicas/read models, partitioning, workload separation and eventually data ownership changes.

186. Scenario / Question: How do you prevent cascading failures?

Expected solution: Timeouts, bounded retries, circuit breakers, bulkheads, queue buffering, load shedding and dependency health monitoring.

187. Scenario / Question: How would you design for graceful degradation in a banking UI?

Expected solution: Keep core transaction paths available, degrade noncritical dashboards/recommendations, show accurate status and never display false transaction success.

188. Scenario / Question: What is the difference between horizontal and vertical scaling?

Expected solution: Vertical adds resources to one instance; horizontal adds instances. Stateless services generally favor horizontal scaling.

189. Scenario / Question: How would you optimize a slow API end to end?

Expected solution: Trace request path, identify top latency contributor, optimize DB/external calls, reduce serialization/payload, cache safe data and tune resource pools, then load-test.

190. Scenario / Question: How do you use caching safely for financial data?

Expected solution: Cache only data whose staleness is acceptable, define TTL/invalidation, avoid sensitive data exposure and never let cache state override transactional truth.

191. Scenario / Question: How would you design a distributed idempotency service?

Expected solution: Use a durable distributed store with unique key constraints, atomic state transitions, TTL/retention policy and consistent response semantics.

192. Scenario / Question: What is eventual consistency and where is it acceptable?

Expected solution: Different replicas/read models converge over time. It may be acceptable for dashboards/search, but core financial balances and transaction commits need stronger consistency.

193. Scenario / Question: How do you handle a traffic spike that exceeds downstream capacity?

Expected solution: Rate limit/load shed, queue asynchronous work, prioritize critical traffic and protect downstream services with concurrency limits.

194. Scenario / Question: How do you choose between synchronous and asynchronous architecture?

Expected solution: Consider user latency, consistency, coupling, failure isolation, throughput and operational complexity.

195. Scenario / Question: How would you design observability for a critical platform?

Expected solution: Metrics, logs and traces tied by correlation IDs, SLO dashboards, business metrics, alert thresholds and actionable runbooks.

196. Scenario / Question: What is a single point of failure in your design?

Expected solution: Any component whose failure stops the required service. Identify them explicitly and add redundancy or graceful degradation.

197. Scenario / Question: How would you test performance before production?

Expected solution: Baseline, load, stress, soak and failure tests using realistic data and traffic patterns; monitor CPU, memory, DB, queues and p95/p99 latency.

198. Scenario / Question: A service is fast at low load but collapses at high load. What does that suggest?

Expected solution: Saturation, contention, nonlinear resource behavior, queue buildup or downstream limits. Look at utilization and queueing rather than just average response time.

199. Scenario / Question: How would you design reusable platform libraries without creating tight coupling?

Expected solution: Small stable interfaces, extension points, semantic versioning, documentation, compatibility testing and avoid leaking internal implementation details.

200. Scenario / Question: How do you balance scalability, availability, consistency and cost?

Expected solution: Start from business SLOs and data correctness requirements; choose the simplest architecture that meets them, then measure and evolve.

11. Financial Crime / Transaction Monitoring / Case Management

20 scenario questions

201. Scenario / Question: A transaction triggers a suspicious-activity rule. How would you design the flow?

Expected solution: Ingest transaction, enrich required data, evaluate rules/risk models, create an alert with traceability, deduplicate where appropriate, then route to case management for investigation.

202. Scenario / Question: The same transaction creates multiple alerts. How would you reduce duplicates?

Expected solution: Define correlation/deduplication keys, rule grouping and time windows; preserve auditability and avoid suppressing genuinely distinct risk signals.

203. Scenario / Question: An investigator opens a case and needs all related transactions. How would you design it?

Expected solution: Use a case ID/entity relationship model and a query-optimized read model/search index where necessary, while retaining authoritative transaction data in the system of record.

204. Scenario / Question: A rule change increases alert volume 20x. What do you do?

Expected solution: Validate rule logic, test against historical data, estimate capacity, introduce controlled rollout, monitor alert quality and tune thresholds with business/risk stakeholders.

205. Scenario / Question: How do you ensure auditability of case decisions?

Expected solution: Record immutable decision events with actor, timestamp, reason, case/alert IDs, previous/new status and correlation information; restrict alteration and monitor access.

206. Scenario / Question: An investigator accidentally closes a high-risk case. How would you prevent this?

Expected solution: Role-based permissions, confirmation for critical actions, segregation of duties where required, workflow controls and immutable audit history.

207. Scenario / Question: How would you handle a long-running case workflow?

Expected solution: Persist workflow state, use asynchronous tasks/events, deadlines/SLA timers and restartable processing rather than holding synchronous HTTP requests.

208. Scenario / Question: A customer disputes an alert. What should the system retain?

Expected solution: The alert evidence, rule/model version, relevant transaction/context, investigator actions, decisions, timestamps and audit trail, subject to retention/privacy policies.

209. Scenario / Question: How would you make case creation idempotent?

Expected solution: Use a deterministic alert/event/business key with a unique constraint and return existing case linkage when duplicate delivery occurs.

210. Scenario / Question: A monitoring system is delayed by 30 minutes. What is your approach?

Expected solution: Measure ingestion/processing lag, inspect Kafka queues, consumer throughput, downstream calls and DB contention; prioritize critical flows and recover backlog safely.

211. Scenario / Question: How do you prevent unauthorized investigators from viewing cases?

Expected solution: Enforce server-side RBAC/ABAC, least privilege, tenant/desk boundaries where applicable, audit access and protect sensitive data in transit/at rest.

212. Scenario / Question: How would you design an alert lifecycle?

Expected solution: Explicit states such as new, assigned, investigating, escalated, resolved/closed, with permitted transitions, actor/audit metadata and idempotent commands.

213. Scenario / Question: A case must be reassigned. What should happen?

Expected solution: Validate authorization, update assignment atomically, create an audit event and notify relevant parties; retain previous ownership history.

214. Scenario / Question: How do you avoid losing an alert during service failure?

Expected solution: Durable event ingestion, transactional state, outbox where needed, consumer idempotency and recovery/replay procedures.

215. Scenario / Question: How would you support regulatory reporting requirements?

Expected solution: Maintain traceable source data, rule/model versions, immutable audit records, retention controls and reproducible reporting processes.

216. Scenario / Question: A rules engine is becoming a performance bottleneck. What can you do?

Expected solution: Profile rule evaluation, pre-filter candidates, cache stable reference data, parallelize safely, partition workloads and scale the engine horizontally.

217. Scenario / Question: How do you handle false positives?

Expected solution: Capture outcome data, analyze rule effectiveness, tune thresholds/rules with risk teams and ensure changes are versioned/tested before deployment.

218. Scenario / Question: How would you test a transaction-monitoring rule?

Expected solution: Use representative historical/synthetic transactions with positive, negative and boundary cases; verify alert creation, suppression/deduplication and audit data.

219. Scenario / Question: How should sensitive financial-crime data be logged?

Expected solution: Never log unnecessary PII/secrets. Use redaction/tokenization, structured metadata, strict access controls and audited log retention.

220. Scenario / Question: What qualities should a Tech Lead bring to a financial-crime platform?

Expected solution: Strong engineering discipline plus risk awareness: correctness, traceability, security, resilience, stakeholder communication, controlled change and measurable operational outcomes.

12. Tech Lead, Agile, Stakeholders & Architecture Governance

20 scenario questions

221. Scenario / Question: Two architects disagree on a design. How do you resolve it?

Expected solution: Clarify requirements and constraints, compare options against measurable criteria, prototype if needed, document the decision and escalate only when decision rights require it.

222. Scenario / Question: A business stakeholder wants a feature in two weeks that normally takes six. What do you do?

Expected solution: Clarify must-have scope, risks and acceptance criteria, propose incremental delivery, estimate transparently and avoid committing to unsafe shortcuts.

223. Scenario / Question: A developer proposes a new framework for a small problem. How do you evaluate it?

Expected solution: Assess business value, operational cost, security, supportability, team expertise, licensing and long-term maintenance; prefer the simplest proven solution.

224. Scenario / Question: How do you run story refinement for a complex API change?

Expected solution: Clarify acceptance criteria, dependencies, NFRs, data/API contracts, security, observability, testing, deployment and rollback; split into deliverable slices.

225. Scenario / Question: A production incident occurs during a sprint. How do you balance delivery and support?

Expected solution: Prioritize business/customer impact, stabilize production, communicate status, re-plan sprint scope transparently and follow with root-cause/remediation work.

226. Scenario / Question: A team repeatedly misses estimates. What do you change?

Expected solution: Inspect refinement quality, story sizing, dependencies and interruptions; use historical throughput and split work rather than pressuring teams to inflate estimates.

227. Scenario / Question: How do you communicate a technical risk to executives?

Expected solution: State impact, probability, affected business outcome, options, recommendation, decision needed and timeline in concise language.

228. Scenario / Question: A team has inconsistent coding standards. How do you drive adoption?

Expected solution: Define lightweight standards, automate formatting/static analysis/quality gates, provide examples and review exceptions rather than policing every line.

229. Scenario / Question: How do you decide whether to refactor or deliver a feature?

Expected solution: Assess risk, change frequency, defect cost and business value. Prefer targeted refactoring that directly reduces delivery/operational risk.

230. Scenario / Question: A critical dependency team misses its commitment. What do you do?

Expected solution: Surface dependency early, identify alternatives/workarounds, re-plan sequence and communicate impact rather than discovering the issue at release time.

231. Scenario / Question: How do you establish API standards across teams?

Expected solution: Publish conventions for naming, errors, versioning, security, pagination, idempotency, observability and documentation; provide reusable libraries/templates.

232. Scenario / Question: How do you review architecture for scalability?

Expected solution: Model expected traffic/data growth, identify bottlenecks and single points of failure, define SLOs and validate assumptions with load tests.

233. Scenario / Question: A senior engineer strongly disagrees with your decision. How do you handle it?

Expected solution: Invite evidence, listen, compare alternatives objectively and document the decision. Once decided, align the team while preserving psychological safety.

234. Scenario / Question: How do you manage technical debt?

Expected solution: Maintain a visible debt backlog, quantify operational/business impact and allocate capacity to high-risk items rather than treating all debt equally.

235. Scenario / Question: What does 'definition of done' mean for a senior engineering team?

Expected solution: Code, tests, review, security/quality checks, documentation where needed, observability, deployability and acceptance criteria completed—not merely code merged.

236. Scenario / Question: How do you mentor engineers without becoming a bottleneck?

Expected solution: Set standards and guardrails, delegate ownership, coach through reviews/design sessions and reserve direct intervention for high-risk decisions.

237. Scenario / Question: How do you ensure cross-platform changes are coordinated?

Expected solution: Map impacted services/contracts, define compatibility sequence, use shared RFC/design docs, dependency tracking and end-to-end validation.

238. Scenario / Question: A stakeholder asks for a shortcut that violates compliance. What do you do?

Expected solution: Explain the risk clearly, propose compliant alternatives and escalate through the proper governance path if necessary; never silently bypass required controls.

239. Scenario / Question: How do you measure success of a technical initiative?

Expected solution: Business outcome plus engineering measures: reliability, latency, deployment frequency, defect rate, cost, adoption and stakeholder satisfaction.

240. Scenario / Question: What would you do in your first 30 days as Tech Lead?

Expected solution: Understand architecture/business flows, meet stakeholders, review production health and delivery process, identify top risks, establish quick wins and create a prioritized technical roadmap.

Final Senior-Level Interview Checklist

- Always state assumptions before designing.
- Talk about scalability, availability, security and observability—not just coding.
- For financial transactions, emphasize idempotency, consistency, auditability and controlled change.
- For microservices, discuss timeouts, retries, circuit breakers, bulkheads and distributed tracing.
- For Kafka, assume duplicate delivery and design idempotent consumers.
- For databases, use execution plans and evidence before tuning.
- For React, optimize based on profiling and keep authorization on the server.
- For CI/CD, build once and promote the same immutable artifact.
- For leadership questions, explain trade-offs, stakeholder communication and measurable outcomes.
- For every production scenario, finish with monitoring, rollback/recovery and regression testing.